Systemy mechatroniczne II
Sterowanie systemów

Wykład 4: Struktury i funkcje języka C/C++, callbacki

Marek Kciuk

Definicje - przypomnienie

- Definicje

```
#define FOREVER 1
#define LEDPORT PORTA
#define LEDOUT DDRA
#define INPORT PINC
#define INPULLUP PORTC
#define DDRIN DDRC
#define PAUSE 300

#define BUTTON 0
```

- Definicje jako makra

```
//Ustawianie bitu w zmiennej
#define SET_BIT(var, bit) ((var) |= _BV(bit))
//Kasowanie bitu
#define CLEAR_BIT(var, bit) ((var) &= ~(1 << (bit)))
//Negacja bitu
#define TOGGLE_BIT(var, bit) ((var) ^= (1 << (bit)))
//Sprawdzenie stanu wysokiego bitu
#define CHECK_BIT(var, bit) ((var) & _BV(bit))
#define BUTTON_RELEASED(var, bit) ((var) & _BV(bit))
//Sprawdzenie stanu niskiego bitu
#define CHECK_BIT_LOW(var, bit) (!((var) & _BV(bit)))
#define BUTTON_PRESSED(var, bit) (!((var) & _BV(bit)))
```

Funkcje – przekazywanie parametrów przez wartość

- Przez wartość tworzona jest kopia zmiennej

```c
#include <stdio.h>
#include <stdint.h>

void MyFun(uint8_t Var)
{
Var = 5;

}


int main()
{
    uint8_t LocalVar = 3;
    MyFun(LocalVar);
    printf("%d", LocalVar);

    return 0;
}
```

Funkcja printf zwraca wartość: 3

Funkcje – przekazywanie parametrów przez wartość

- Przez wartość tworzona jest kopia zmiennej

```c
#include <stdio.h>
#include <stdint.h>

uint8_t MyFun(uint8_t Var)
{
    Var = 5;
    return Var;
}


int main()
{
    uint8_t LocalVar = 3;
    LocalVar = MyFun(LocalVar);
    printf("%d", LocalVar);

    return 0;
}
```

Funkcja printf zwraca wartość: 5

Funkcje – przekazywanie parametrów przez wskaźnik

- Przez wskaźnik przekazywany jest adres komórki

```
#include <stdio.h>
#include <stdint.h>

void MyFun(uint8_t* Var)
{
    *Var = 5;
}


int main()
{
    uint8_t LocalVar = 3;
    MyFun(&LocalVar);
    printf("%d", LocalVar);

    return 0;
}
```

Funkcja printf zwraca wartość: 5

Bezpośrednie przekazanie adresu zmiennej jako parametru wejściowego funkcji

Funkcje – przekazywanie parametrów przez wskaźnik

- Przez wskaźnik przekazywany jest adres komórki

```
uint8_t* VarAddr;


void MyFun(uint8_t* Var)
{
    *Var = 5;
}

int main()
{
    uint8_t LocalVar = 3;
    VarAddr = &LocalVar;

    MyFun(VarAddr);
    printf("%d", LocalVar);

    return 0;
}
```

Zmienna wskaźnikowa (wskazuje na typ danych)

Funkcja printf zwraca wartość: 5

Wyłuskanie adresu zmiennej LocalVar do zmiennej VarAddr

Wywołanie funkcji z parametrem w postaci zmiennej wskaźnikowej

Typ wyliczeniowy - enum

- Tworzy tekstowe „etykiety” dla liczb całkowitych,
- Używamy jak zwykłej zmiennej,
- Domyślnie pierwsza etykieta ma wartość 0,

```
enum <nazwa>
{
Etykieta1,
Etykieta2,
Etykieta3,
};
```

```
enum <tydzien>
{
Poniedzialek,
Wtorek,
Sroda,
Czwartek,
Piatek,
Sobota,
Niedziela
};
```

```
enum ErrorCode
{
NO_ERROR,
WRONG_VALUE,
NO_FREE_SPACE
};
```

Utworzenie zmiennej ErrorVar jako instancji enum ErrorCode

```
enum ErrorCode ErrorVar;
```

enum ErrorCode jako element zwracany przez funkcję

```
enum ErrorCode fun(void)
{
…
return NO_ERROR;
}
```


Typ wyliczeniowy - enum

- Kolejne wartości są inkrementowane:
  - Wtorek = 2
  - Środa = 3
  - …
  - Niedziela = 12

```
enum <tydzien>
{
Poniedzialek = 1,
Wtorek,
Sroda,
Czwartek,
Piatek,
Sobota = 11,
Niedziela
};
```

Typ wyliczeniowy - enum

- Enum do zwiększenia czytelności kodu – if i switch

```
if(Error == WRONG_VALUE)
{
// Error reaction
}
```

```
switch(Error)
{
    case NO_ERROR:
        // NO_ERROR code
    break;
    case WRONG_VALUE:
        // WRONG_VALUE code
    break;
    case NO_FREE_SPACE:
        // NO_FREE_SPACE code
    break;
    default: //* opcjonalny, w arduino obowiązkowy

    break
}
```


Definicja typu - typedef

- Tworzy definicję typu na podstawie już istniejącej
- Przykład jak zdefiniowano typ uint8_t

```
typedef <źródło> <nowy typ>;

typedef char uint8_t;
typedef unsigned int uint32_t;
```

Od tego momentu char i uint8_t to to samo... Ale kompilator zwróci błąd lub warning, jeśli je pomylimy...

- ErrorCode jako definicja typu

```
typedef enum
{
NO_ERROR,
WRONG_VALUE,
NO_FREE_SPACE
}ErrorCode_t;
```

Dzięki typedef deklarujemy jak każdą inną zmienną:

```
ErrorCode_t ErrorVar;
```

Nazwa typu enumeracyjnego

Zamiast wersji wcześniejszej: `enum ErrorCode ErrorVar;`

Złożone typy danych - struktury

- Struktura to „pudło” będące kolekcją różnych typów danych,

```
struct <nazwa> {
    Typ1 Pole1;
    Typ2 Pole2;
    Typ3 Pole3;
};

struct MyStruct
{
    uint8_t VarUint8;
    uint16_t VarUint16;
    char Sign;
    uint32_t RegisterValue;
};
```

```
typedef struct
{
    uint8_t VarUint8;
    uint16_t VarUint16;
    char Sign;
    uint32_t RegisterValue;
} MyStruct_t;
```

Utworzenie zmiennej:

```
struct MyStruct VarStruct;
```

```
MyStruct_t VarStruct;
```


Złożone typy danych - struktury

- Odwołanie do pól struktury - kropka,

Funkcja z przekazaniem zmiennej przez wartość

```
MyStruct_t VarStruct;

VarStruct.VarUint8 = 33;
VarStruct.Sign = 'a';

if(VarStruct.Sign == 'x')...
```

Funkcja printf zwraca warość: 9, 0, a

```
MyStruct_t MyFun(MyStruct_t Var)
{
    Var.VarUint8 = Var.VarUint8 + 5;
    Var.Sign = 'a';
    return Var;
}


int main()
{
    MyStruct_t LocalVar;
    LocalVar.VarUint8 = 4;
    LocalVar.VarUint16 = 0;
    LocalVar.Sign = 'X';

    LocalVar = MyFun(LocalVar);

    printf("%d, %d, %c", LocalVar.VarUint8,
    LocalVar.VarUint16, LocalVar.Sign);

    return 0;
}
```

Złożone typy danych - struktury

- Odwołanie do pól struktury przekazanej przez wskaźnik „->”

Funkcja z przekazaniem zmiennej przez wskaźnik

Odwołanie do pola w strukturze

Wyłuskanie adresu zmiennej typu strukturalnego

Funkcja printf zwraca warość: 9, 0, a

```
void MyFun(MyStruct_t* Var)
{
    Var->VarUint8 = Var->VarUint8 + 5;
    Var->Sign = 'a';
}

int main()
{
    MyStruct_t LocalVar;
    LocalVar.VarUint8 = 4;
    LocalVar.VarUint16 = 0;
    LocalVar.Sign = 'X';

    MyFun(&LocalVar);

    printf("%d, %d, %c", LocalVar.VarUint8,
    LocalVar.VarUint16, LocalVar.Sign);

    return 0;
}
```


Złożone typy danych - struktury

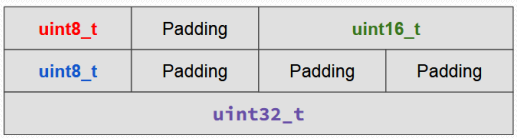
- Ile miejsca zajmuje struktura?

```
typedef struct {
    uint8_t Age;
    uint16_t Height;
    uint8_t Weight;
} Human_t;
```



Teoretycznie powinna 4 bajty a jest 6 bajtów

```
typedef struct {
    uint8_t Age;
    uint16_t Height;
    uint8_t Weight;
    uint32_t AgeInDays;
} Human_t;
```



Teoretycznie powinna 8 bajtów a jest 12 bajtów

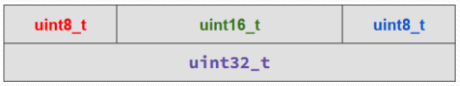
Złożone typy danych - struktury

- Jak poprawić pakowanie struktury?

- Dyrektywa #pragma pack(1)

```
#pragma pack(1)
typedef struct {
    uint8_t Age;
    uint16_t Height;
    uint8_t Weight;
    uint32_t AgeInDays;
} Human_t;
```

Tutaj pragma jest „globalna” dla wszystkich struktur



Teoretycznie powinna 8 bajtów
i jest 8 bajtów

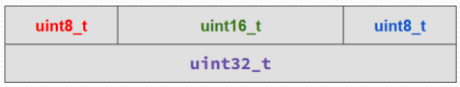
Złożone typy danych - struktury

- Jak poprawić pakowanie struktury?

- Dyrektywa #pragma pack(1)

Tutaj pragma jest jako parametr dla konkretnej struktury

```
typedef struct __attribute__((packed)){
    uint8_t Age;
    uint16_t Height;
    uint8_t Weight;
    uint32_t AgeInDays;
} Human_t;
```



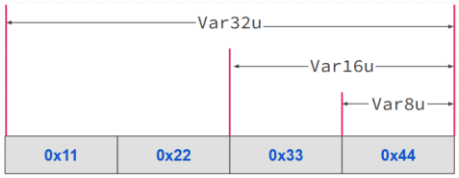
Teoretycznie powinna 8 bajtów
i jest 8 bajtów

Złożone typy danych - unie

- Unia to „pomieszanie” różnych typów danych,
- Rozmiar unii wynosi tyle ile jej największy element,
- Wszystkie zmienne zaczynają się od tej samej komórki,
- Służy głównie do konwersji typów,

```
typedef union
{
    uint8_t Var8u;
    uint16_t Var16u;
    uint32_t Var32u;
} MyUnion_t;

MyUnion_t UnionVar;
VarUnion.Var32u = 0x11223344
```



Złożone typy danych - unie

- Konwersja typów - przykłady

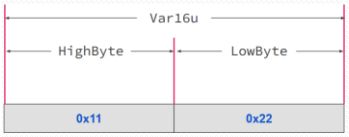
Unia zawierająca tablicę

```
typedef union
{
    uint8_t Byte[2];
    uint16_t Var16u;
} Conv16to8_t;
```

Unia zawierająca strukturę (bez nazwy)

```
typedef union
{
    struct
    {
        uint8_t LowByte;
        uint8_t HighByte;
    };
    uint16_t Var16u;
} Conv16to8_t;

Conv16to8_t Converter;
Converter.Var16u = 0x1122;
```



Złożone typy danych - unie

- Konwersja typów - przykłady

Unia zawierająca tablicę

```
typedef union
{
    uint8_t Byte[2];
    uint16_t Var16u;
} Conv16to8_t;

int main()
{
    Conv16to8_t Converter;
    Converter.Var16u = 0x1122;

    printf("0x%X, 0x%X", Converter.
    Byte[1], Converter.Byte[0]);

    return 0;
}
```

Łatwiej inkrementować w pętli for wysyłającej do pamięci

Unia zawierająca strukturę (bez nazwy)

```
typedef union
{
    struct
    {
        uint8_t LowByte;
        uint8_t HighByte;
    };
    uint16_t Var16u;
} Conv16to8_t;


int main()
{
    Conv16to8_t Converter;
    Converter.Var16u = 0x1122;

    printf("0x%X, 0x%X", Converter.HighByte,
    Converter.LowByte);

    return 0;
}
```


Nazwa funkcji jako parametr wejściowy funkcji

- Funkcja także ma adres w pamięci, więc możemy przekazać ją podobnie jak wskaźnik do zmiennej

UWAGA! Wszystkie funkcje muszą mieć **TAKĄ SAMĄ** strukturę
* Jest adresem funkcji

```
void word(void);
void hello(void);

void Say(void(*Fun)(void));

int main()
{
    Say(hello);
    Say(word);

    return 0;
}
```

Funkcje wywoływane

Funkcje wywołująca

Przekazanie funkcji jako parametru wejściowego

```
void Say(void(*Fun)(void))
{
    Fun();
}

void word(void)
{
    printf("Word\n");
}

void hello(void)
{
    printf("Hallo\n");
}
```

Nazwa funkcji jako parametr wejściowy funkcji

- Funkcja także ma adres w pamięci, więc możemy przekazać ją podobnie jak wskaźnik do zmiennej

UWAGA! Wszystkie funkcje muszą mieć **TAKĄ SAMĄ** strukturę
* - jest adresem funkcji

```
void word(void);
void hello(void);

void Say(void(*Fun)(void));

int main()
{
    Say(hello);
    Say(word);

    return 0;
}
```

Funkcje wywoływane

Funkcje wywołująca

Przekazanie funkcji jako parametru wejściowego

Terminal zwrócił:

```
Hallo
Word
```

```
void Say(void(*Fun)(void))
{
    Fun();
}

void word(void)
{
    printf("Word\n");
}

void hello(void)
{
    printf("Hallo\n");
}
```

11:59 21 z 34

Callback – rejestracja funkcji przed wywołaniem

Funkcje word() i hello() identyczne jak na poprzednim slajdzie

Zmienna typu wskaźnik na funkcję

Funkcje i ich przekazanie – wszystkie muszą mieć TĄ SAMĄ strukturę co wskaźnik

```
void (*funPtr)(void);

void word(void);
void hello(void);

void FunctionRegister(void(*Fun)(void));

void CallFunction(void);

int main()
{
    FunctionRegister(hello);
    CallFunction();

    FunctionRegister(word);
    CallFunction();

    return 0;
}
```

Rejestracja funkcji – wpisanie adresu funkcji do zmiennej wskaźnikowej

```
void FunctionRegister(void(*Fun)(void))
{
    if(Fun != NULL)
    {
        funPtr = Fun;
    }
}
```

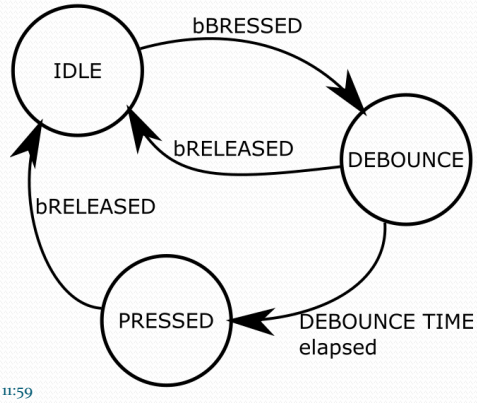
Wywołanie funkcji przez jej wskaźnik

```
void CallFunction(void)
{
    funPtr();
}
```

11:59 22 z 34

Maszyna stanów do obsługi przycisków z callbackiem

- Struktura maszyny stanów do obsługi przycisku




```
typedef enum
{
  IDLE = 0,
  DEBOUNCE,
  PRESSED
} BUTTON_STATE;
```


Maszyna stanów do obsługi przycisków z callbackiem

- „Obiekt” obsługujący przycisk

State – aktualny stan przycisku,
Switch – dane przycisku (adres),
LastTick – timestamp ostatniej zmiany stanu,

```
typedef struct
{
  BUTTON_STATE  State;
  uint8_t       Switch;
  long LastTick;
  long TimerDebounce; //fixed (max value)

  void(*ButtonPressed)(void); //funkcja dla callbacka
} Button t;
```

TimerDebounce – czas trwania stanu debounce,

ButtonPressed – wskaźnik na funkcję obsługi naciśnięcia przycisku (callback)

Maszyna stanów do obsługi przycisków z callbackiem

- Funkcje interfejsowe – dostępne w programie main():

```
//public button
void ButtonInitKey(Button_t* Key, uint8_t SwitchPin, long Debounce);
void ButtonTask(Button_t* Key);
void ButtonCalbackRegister(Button_t* Key, void (*Callback)(void));
```

ButtonInitKey – funkcja inicjująca obiekt typu **przycisk** ze wskazaniem na wcześniej utworzony obiekt za pomocą wskaźnika,

ButtonTask – główna funkcja obsługująca **przycisk**,

ButtonCallbackRegister – funkcja rejestrująca callback – przekazująca wskaźnik na funkcję do obiektu **przycisk**

Maszyna stanów do obsługi przycisków z callbackiem

- Funkcje obsługi maszyny stanów:

```
//state routines
void ButtonIdleRoutine(Button_t* Key);
void ButtonDebounceRoutine(Button_t* Key);
void ButtonPressedRoutine(Button_t* Key);
```

Maszyna stanów do obsługi przycisków z callbackiem

- Inicjacja i rejestracja przycisku:

```
// Button init
void ButtonInitKey(Button_t* Key, uint8_t SwitchPin, long Debounce)
{
  Key ->State = IDLE;
  Key ->Switch = SwitchPin;
  Key ->TimerDebounce = Debounce;
}

// Button calback registration
void ButtonCalbackRegister(Button_t* Key, void (*Callback)(void))
{
  if(Callback != NULL)
  {
    Key ->ButtonPressed = Callback;
  }
}
```


Maszyna stanów do obsługi przycisków z callbackiem

- Obsługa maszyny stanów – ButtonTask():

```
// state machine
void ButtonTask(Button_t* Key)
{
  switch(Key->State)
  {
    case IDLE:
      ButtonIdleRoutine(Key);
    break;
    case DEBOUNCE:
      ButtonDebounceRoutine(Key);
    break;
    case PRESSED:
      ButtonPressedRoutine(Key);
    break;
    default:

    break;
  }
}
```

Podano wskaźnik jako wejście więc dalej nie trzeba gwiazdek

Maszyna stanów do obsługi przycisków z callbackiem

- Inicjacja i rejestracja przycisku:

```
// Button init
void ButtonInitKey(Button_t* Key, uint8_t SwitchPin, long Debounce)
{
  Key ->State = IDLE;
  Key ->Switch = SwitchPin;
  Key ->TimerDebounce = Debounce;
}

// Button calback registration
void ButtonCalbackRegister(Button_t* Key, void (*Callback)(void))
{
  if(Callback != NULL)
  {
    Key ->ButtonPressed = Callback;
  }
}
```


Maszyna stanów do obsługi przycisków z callbackiem

- Działanie programu

Utworzenie obiektu typu Button_t

Konfiguracja obiektu BlueKey

Pierwsza rejestracja callbacka w obiekcie BlueKey

Można zmienić callback w dowolnym momencie

Obsługa przycisku w pętli

```
void Led_13_Switch(void)
{
  digitalWrite(13, !digitalRead(13));
}

Button_t BlueKey;

void setup()
{
  // put your setup code here, to run once:
  ButtonInitKey(&BlueKey,7,40);
  pinMode(7, INPUT_PULLUP);
  pinMode(13,OUTPUT);
  digitalWrite(13,LOW);
  ButtonCalbackRegister(&BlueKey, Led_13_Switch);
  Serial.begin(9600);
}

void loop()
{
  // put your main code here, to run repeatedly:
  ButtonTask(&BlueKey);
}
```

Maszyna stanów do obsługi przycisków z callbackiem

- Stan IDLE

Jeśli wciśnięto przycisk to:
Zapamiętaj aktualny tick
Idź do stanu DEBOUNCE

```
void ButtonIdleRoutine(Button_t* Key)
{
  if(!digitalRead(Key->Switch))
  {
    Key->State = DEBOUNCE;
    Key->LastTick = millis();
  }
}
```


Maszyna stanów do obsługi przycisków z callbackiem

- Stan DEBOUNCE

Kiedy upłynął czas TimerDebounce

Sprawdź ponownie przycisk

Jeśli nadal wciśnięty
Wywołaj funkcję callback
I przejdź do stanu PRESSED

Jeśli przycisk zwolniony
Wróć do IDLE

```
void ButtonDebounceRoutine(Button_t* Key)
{
  if(millis() - Key->LastTick <= Key->TimerDebounce)
  {
    // debounce time expired
    if(!digitalRead(Key->Switch))
    {
      // button is still pressed
      Key->State = PRESSED;
      if(Key->ButtonPressed != NULL)
      {
        Key->ButtonPressed();
      }
    }else
    {
      // button is released
      Key->State = IDLE;
    }
  }
}
```

Maszyna stanów do obsługi przycisków z callbackiem

- Stan PRESSED

Czekaj na zwolnienie przycisku

```
void ButtonPressedRoutine(Button_t* Key)
{
  if(digitalRead(Key->Switch))
  {
    Key->State = IDLE;
  }
}
```

Można również zrobić opóźnienie, żeby nie sprawdzać co iteracją podobnie jak w IDLE

Dziękuję za uwagę

Bibliografia:
Materiały do kursu: Mateusz Salamon: Język C dla mikrokontrolerów